



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

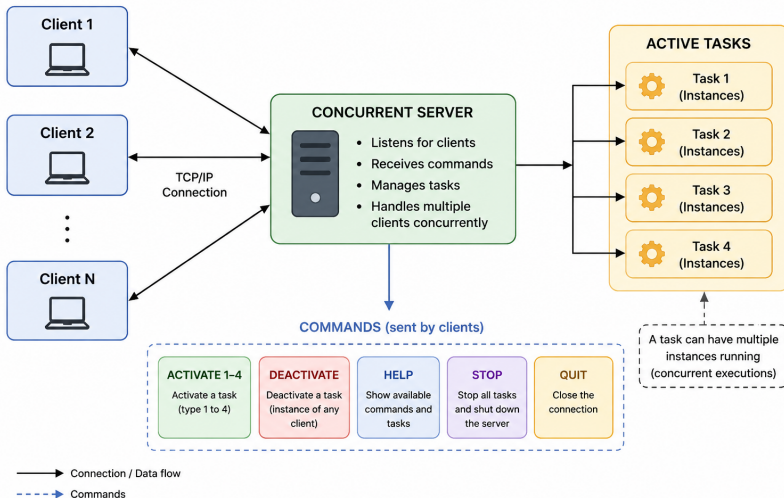
Concurrent Server TCP/IP

Concurrent Real Time Programming

Furlan Davide

August 20, 2026

- **Objective:** Simulate a dynamic environment where periodic tasks are activated/deactivated via a remote supervisor (Client) through a central controller (Server).
- **System Model:**
 - **Server:** Manages a pool of clients and a pool of real-time worker threads.
 - **Clients:** Issue commands to modify the system state (Activate, Deactivate, Block, Stop, Quit).
 - **Tasks:** Periodic routines with predefined execution times and periods.
- **The Challenge:** Managing high-frequency state changes across multiple threads (Connection Handlers, Task Workers, and the Main Server Loop) without incurring race conditions or deadlocks.



- **Decoupled Logic:** The server separates connection management from task execution.
- **Concurrency Model:** One thread per client (Connection Handler) and one thread per active task (Worker Thread).

To ensure thread safety, the system relies on two primary shared arrays:

1. Client `clients[MAX_THREADS]`:
 - Tracks connected clients, their socket FDs, and assigned ports.
 - **Protected by:** `mutex_clients_array`, `roomAvailable`.
2. ActiveTask `tasks_active[MAX_NUMBER_ACTIVE_TASKS]`:
 - Tracks currently running periodic tasks, their IDs, and thread handles.
 - **Protected by:** `active_tasks_mutex`.

Note: These structures represent the "Critical Sections" where race conditions are most likely to occur during simultaneous command processing.

When a client sends an ACTIVATE <ID> command, the Server follows this protocol:

1. **Parsing:** Extract task ID from the command string.
2. **Schedulability Check:** Run `is_schedulable()`. This is a read-only operation on the task array.
3. **Resource Acquisition:**
 - Attempt to decrement the `free_slots_sem`.
 - Lock `active_tasks_mutex`.
 - Locate an empty slot in `tasks_active`.
4. **Thread Spawning:** Call `pthread_create` for `handling_active_task`.
5. **Unlock:** Release `active_tasks_mutex`.

Command Logic: BLOCK, QUIT, and STOP



- **BLOCK (Deactivate Task):**
 - Finds the specific instance in `tasks_active`.
 - Sets field of that task `active = 0` inside a `active_tasks_mutex` block.
 - **Crucial:** The worker thread, seeing `active == 0`, terminates its loop.
 - Calls `sem_post` to signal a slot is now free.
- **QUIT (Client Disconnect):**
 - Closes the socket and calls `rmvclient()` which...
 - Uses `mutex_clients_array` to update the client buffer.
- **STOP (Global Shutdown):**
 - Updates the global `close_server` flag.
 - **Synchronization:** Uses `mutex_stopserver` to ensure atomic visibility of the shutdown signal to all threads.

Synchronization Strategy I:

Mutexes



The implementation utilizes several `pthread_mutex_t` objects to enforce Mutual Exclusion:

`mutex_clients_array` Protects the integrity of the client registry. Ensures only one thread modifies the client list at a time.

`active_tasks_mutex` Protects the task registry. Critical for preventing corruption when tasks are added or removed.

`mutex_stopserver` Protects the global shutdown flag, preventing a race between the "Stop" command and the main loop's exit condition.

`log_mutex` **Output Integrity:** Essential for real-time debugging. Prevents multiple threads from interleaving their `printf` calls, which would render the server logs unreadable.

Synchronization Strategy II:

Semaphores



The system uses a **Counting Semaphore** (`free_slots_sem`) to manage the `tasks_active` array:

- **Initialization:** `sem_init(&free_slots_sem, 0, MAX_NUMBER_ACTIVE_TASKS)`.
- **Resource Tracking:** The semaphore acts as a counter for available slots.
- **Non-Blocking Allocation:**
 - `sem_trywait` is used during activation.
 - If the semaphore is 0 (no slots), the server immediately rejects the task with a FULL CAPACITY message instead of hanging.
- **Reclamation:** When a task is de-activated, `sem_post` increments the counter, allowing the next ACTIVATE command to proceed.

Synchronization Strategy III:

Condition Variables



To handle the “Server Full” state for clients, we employ `pthread_cond_t (roomAvailable)`:

- **The Consumer (Connection Handler):** If `findFreePosition()` returns `-1` (Server’s array for client is full), the thread enters:
 - `pthread_mutex_lock(&mutex_clients_array)`
 - `pthread_cond_wait(&roomAvailable, &mutex_clients_array)`
- **The Signal:** When `rmvclient()` is called (a client disconnects), `pthread_cond_signal(&roomAvailable)` is invoked, waking up a waiting thread so it can occupy the newly available slot.

Summary of Synchronization Primitives



Primitive	Variable	Purpose
Mutex	mutex_clients_array	Client array integrity
Mutex	active_tasks_mutex	Task array integrity
Mutex	log_mutex	Thread-safe I/O (Console)
Mutex	mutex_stopserver	Atomic shutdown flag
Semaphore	free_slots_sem	Counting available task slots
Cond Var	roomAvailable	Managing client connection overflow

Table: Mapping of primitives to system constraints.

- **Robustness:** The system successfully bridges the gap between high-level TCP commands and low-level real-time thread management.
- **Concurrency Safety:** By utilizing such synchronization approach (Mutexes for state, Semaphores for resources, and Condition Variables for flow control), the system avoids the classic pitfalls of multi-threaded C programming.
- **Scalability:** The architecture allows for N clients and M tasks to operate simultaneously, governed strictly by the defined synchronization boundaries.

Questions?

- Compare Rate Monotonic with alternative fixed-priority policies.
- Extend evaluation to dynamic-priority algorithms (e.g., EDF).
- Evaluate system performance under high CPU utilization and overloaded conditions.
- Thread pool for clients architecture.
- Possibly constant scan of all the clients and active tasks